

Algoritmi e Strutture Dati

Dispensa riassuntiva: algoritmi principali

Paolo Bianchessi

5 febbraio 2026

Indice

1 Guida alla lettura	2
2 Ricerca in array ordinati	2
2.1 Ricerca dicotomica (Binary Search)	2
3 Ordinamento basato su confronti	2
3.1 Selection Sort	3
3.2 Insertion Sort	3
3.3 Bubble Sort (base e ottimizzato)	4
3.4 Merge Sort	4
3.5 Quick Sort	5
3.6 Heap Sort	6
4 Ordinamento senza confronti	6
4.1 IntegerSort (conteggio)	6
4.2 BucketSort	7
4.3 RadixSort (LSD, base B)	7
5 Visite di grafi e alberi	8
5.1 BFS (visita in ampiezza)	8
5.2 DFS (visita in profondità)	8
5.3 Visite di alberi (pre/in/post-order, level-order)	9
6 Minimo albero ricoprente (MST)	9
6.1 Kruskal	10
6.2 Prim	10
7 Cammini minimi nei grafi	11
7.1 Dijkstra (da una sorgente)	11
7.2 Bellman–Ford (da una sorgente)	12
7.3 Floyd–Warshall (tutte le coppie)	13
8 Algoritmi non deterministici per problemi NP	13
8.1 Schema generale “indovina e verifica”	13
8.2 Partizione (Partition)	13
8.3 Clique di dimensione k	14

1 Guida alla lettura

Questa dispensa raccoglie i principali algoritmi visti nel corso, organizzati per argomento. Per *ogni* algoritmo sono presenti due versioni:

- **Dettaglio:** obiettivo, idea, pseudocodice, correttezza (intuizione), complessità e note d'uso.
- **Mappa mentale:** una scheda ultra-rapida per ripasso.

2 Ricerca in array ordinati

Assumiamo un array ordinato $A[0 \dots n-1]$ e una chiave x da cercare.

2.1 Ricerca dicotomica (Binary Search)

Dettaglio

Idea. Confronta x con l'elemento centrale dell'intervallo corrente e dimezza ogni volta lo spazio di ricerca. **Precondizione.** A ordinato in senso non decrescente.

Algorithm 1 Ricerca dicotomica (iterativa)

```
1: procedure BINARYSEARCH( $A, x$ )
2:    $l \leftarrow 0; r \leftarrow n - 1$ 
3:   while  $l \leq r$  do
4:      $m \leftarrow \lfloor (l + r)/2 \rfloor$ 
5:     if  $A[m] = x$  then
6:       return  $m$ 
7:     if  $A[m] < x$  then
8:        $l \leftarrow m + 1$ 
9:     else
10:       $r \leftarrow m - 1$ 
11:   return NIL
```

Complessità. Numero di confronti $\Theta(\log n)$; spazio extra $\Theta(1)$. **Nota.** Su array ordinati, la ricerca è logaritmica, ma inserimento/cancellazione interna restano $\Theta(n)$ per gli shift.

Mappa mentale

- **Richiede:** array ordinato.
- **Invariante:** x (se presente) sta in $A[l..r]$.
- **Passo:** confronta con $A[m] \rightarrow$ dimezza intervallo.
- **Costo:** $\Theta(\log n)$ confronti, $\Theta(1)$ spazio.

3 Ordinamento basato su confronti

Assumiamo un array $A[0 \dots n-1]$ e una relazione di ordine totale $\leq$ sulle chiavi.

3.1 Selection Sort

Dettaglio

Idea. Mantieni un prefisso ordinato; al passo k scegli il minimo di $A[k \dots n - 1]$ e scambialo in posizione k .

Algorithm 2 SelectionSort

```
1: procedure SELECTIONSORT( $A$ )
2:   for  $k \leftarrow 0$  to  $n - 2$  do
3:      $m \leftarrow k$ 
4:     for  $j \leftarrow k + 1$  to  $n - 1$  do
5:       if  $A[j] < A[m]$  then
6:          $m \leftarrow j$ 
7:     scambia  $A[k]$  e  $A[m]$ 
```

Complessità. Confronti sempre $\sum_{k=0}^{n-2} (n - k - 1) = \Theta(n^2)$. Scambi $\leq n$. **Proprietà.** In-place, *non* stabile.

Mappa mentale

- **Quando:** semplice, pochi scambi (memorie lente / scritture costose).
- **Come:** per $k = 0..n - 2 \rightarrow$ minimo in coda $\rightarrow$ swap.
- **Costo:** $\Theta(n^2)$ confronti, $\mathcal{O}(n)$ scambi.
- **Note:** non stabile.

3.2 Insertion Sort

Dettaglio

Idea. Il prefisso $A[0 \dots k - 1]$ è ordinato; inserisci $A[k]$ spostando a destra gli elementi più grandi.

Algorithm 3 InsertionSort

```
1: procedure INSERTIONSORT( $A$ )
2:   for  $k \leftarrow 1$  to  $n - 1$  do
3:      $x \leftarrow A[k]$ 
4:      $j \leftarrow k - 1$ 
5:     while  $j \geq 0$  and  $A[j] > x$  do
6:        $A[j + 1] \leftarrow A[j]$ 
7:        $j \leftarrow j - 1$ 
8:      $A[j + 1] \leftarrow x$ 
```

Complessità. Peggior $\Theta(n^2)$; miglior $\Theta(n)$ (array già ordinato); in generale $\Theta(n^2)$ su input casuale. **Proprietà.** In-place, stabile.

Mappa mentale

- **Quando:** array quasi ordinati; n piccolo; come sottopasso di algoritmi ibridi.
- **Come:** prendo $x = A[k]$ e lo “inserisco” nel prefisso shiftando.

- **Costo:** best $\Theta(n)$, worst $\Theta(n^2)$.
- **Note:** stabile, in-place.

3.3 Bubble Sort (base e ottimizzato)

Dettaglio

Idea. Confronta coppie adiacenti e scambia se fuori ordine; ripeti passate finché non si osservano scambi.

Algorithm 4 BubbleSort (con stop anticipato)

```

1: procedure BUBBLESORT( $A$ )
2:   for  $i \leftarrow 0$  to  $n - 2$  do
3:      $swapped \leftarrow \text{false}$ 
4:     for  $j \leftarrow 0$  to  $n - 2 - i$  do
5:       if  $A[j] > A[j + 1]$  then
6:         scambia  $A[j], A[j + 1]$ 
7:          $swapped \leftarrow \text{true}$ 
8:     if not  $swapped$  then
9:       break

```

Complessità. Worst $\Theta(n^2)$; best $\Theta(n)$ con stop anticipato. **Proprietà.** In-place, stabile.

Mappa mentale

- **Quando:** quasi mai per prestazioni; utile didatticamente.
- **Come:** passate con swap adiacenti; se nessuno swap $\rightarrow$ finito.
- **Costo:** worst $\Theta(n^2)$, best $\Theta(n)$.
- **Note:** stabile, in-place.

3.4 Merge Sort

Dettaglio

Idea (divide et impera). Dividi l'array a metà, ordina ricorsivamente le due metà, poi *fai merge* in modo lineare.

Algorithm 5 MergeSort (schema)

```

1: procedure MERGESORT( $A$ )
2:   if  $|A| \leq 1$  then
3:     return
4:   dividi  $A$  in  $B$  e  $C$  (metà/metà)
5:   MERGESORT( $B$ ); MERGESORT( $C$ )
6:    $A \leftarrow \text{MERGE}(B, C)$ 

```

Algorithm 6 Merge

```
1: function MERGE( $B, C$ )
2:   crea array  $X$  di lunghezza  $|B| + |C|$ 
3:    $i \leftarrow 0, j \leftarrow 0, k \leftarrow 0$ 
4:   while  $i < |B|$  and  $j < |C|$  do
5:     if  $B[i] \leq C[j]$  then
6:        $X[k] \leftarrow B[i]; i \leftarrow i + 1$ 
7:     else
8:        $X[k] \leftarrow C[j]; j \leftarrow j + 1$ 
9:      $k \leftarrow k + 1$ 
10:  while  $i < |B|$  do
11:     $X[k] \leftarrow B[i]; i \leftarrow i + 1; k \leftarrow k + 1$ 
12:  while  $j < |C|$  do
13:     $X[k] \leftarrow C[j]; j \leftarrow j + 1; k \leftarrow k + 1$ 
14:  return  $X$ 
```

Complessità. $T(n) = 2T(n/2) + \Theta(n)$ quindi $T(n) = \Theta(n \log n)$. **Spazio.** Tipicamente $\Theta(n)$ extra per gli array di appoggio. **Proprietà.** Stabile; non in-place (nella versione standard).

Mappa mentale

- **Quando:** ordinamento stabile; dati grandi; costi prevedibili.
- **Come:** dividi $\rightarrow$ ordina $\rightarrow$ merge lineare.
- **Costo:** sempre $\Theta(n \log n)$, spazio $\Theta(n)$.
- **Note:** stabile; ottimo worst-case.

3.5 Quick Sort

Dettaglio

Idea. Scegli un perno, partiziona l'array in elementi $\leq$ perno e $>$ perno, ordina ricorsivamente le due parti.

Algorithm 7 QuickSort (in-place, schema)

```
1: procedure QUICKSORT( $A, \ell, r$ )
2:   if  $\ell \geq r$  then
3:     return
4:    $p \leftarrow \text{PARTITION}(A, \ell, r)$ 
5:   QUICKSORT( $A, \ell, p - 1$ )
6:   QUICKSORT( $A, p + 1, r$ )
```

Partizionamento (intuizione). Due indici scorrono da sinistra e da destra: si scambiano gli elementi “fuori lato” finché gli indici si incontrano; il perno finisce nella sua posizione finale.

Complessità. Best/avg $\Theta(n \log n)$; worst $\Theta(n^2)$ (partizioni molto sbilanciate). **Spazio.** $\mathcal{O}(\log n)$ stack in media; $\mathcal{O}(n)$ worst. **Proprietà.** In-place; non stabile.

Mappa mentale

- **Quando:** velocissimo in pratica; in-place.
- **Come:** pivot $\rightarrow$ partition $\rightarrow$ ricorsione su due lati.
- **Costo:** avg $\Theta(n \log n)$, worst $\Theta(n^2)$.
- **Trucchi:** scelta pivot (random/mediana), eliminazione ricorsione in coda.
- **Note:** non stabile.

3.6 Heap Sort

Dettaglio

Idea. Rappresenta un *max-heap* nell'array. Ripeti: scambia radice (massimo) con l'ultimo elemento non ancora sistemato, riduci la dimensione dello heap e ripristina la proprietà di heap (**fixHeap**).

Algorithm 8 HeapSort (schema)

```

1: procedure HEAPSORT( $A$ )
2:   BUILDMAXHEAP( $A$ )  $\triangleright \Theta(n)$ 
3:   for  $end \leftarrow n - 1$  downto 1 do
4:     scambia  $A[0]$  e  $A[end]$ 
5:     FIXHEAP( $A, 0, end - 1$ )

```

Complessità. Build $\Theta(n)$, poi $n - 1$ chiamate a **FixHeap** da $\mathcal{O}(\log n)$: totale $\Theta(n \log n)$.
Proprietà. In-place; non stabile; worst-case $\Theta(n \log n)$.

Mappa mentale

- **Quando:** worst-case garantito $\Theta(n \log n)$ e in-place.
- **Come:** build heap $\rightarrow$ estrai max (swap) $\rightarrow$ fix.
- **Costo:** $\Theta(n \log n)$; spazio $\mathcal{O}(1)$ extra.
- **Note:** non stabile; access pattern meno cache-friendly di quick/merge.

4 Ordinamento senza confronti

Qui le chiavi sono intere (o riducibili a cifre), e sfruttiamo il dominio delle chiavi.

4.1 IntegerSort (conteggio)

Dettaglio

Assunzione. Gli elementi sono interi in $[0, b - 1]$. **Idea.** Conta quante volte appare ogni valore, poi ricostruisci l'array in ordine.

Algorithm 9 IntegerSort (Counting)

```
1: procedure INTEGERSORT( $A, b$ )
2:   crea  $Y[0 \dots b-1]$  e inizializza a 0
3:   for  $i \leftarrow 0$  to  $n-1$  do
4:      $Y[A[i]] \leftarrow Y[A[i]] + 1$ 
5:    $j \leftarrow 0$ 
6:   for  $x \leftarrow 0$  to  $b-1$  do
7:     for  $h \leftarrow 1$  to  $Y[x]$  do
8:        $A[j] \leftarrow x; j \leftarrow j + 1$ 
```

Complessità. $\Theta(n+b)$ tempo, $\Theta(b)$ spazio. **Note.** Non usa confronti; efficace se b non è troppo grande rispetto a n .

Mappa mentale

- **Quando:** chiavi in range piccolo/medio.
- **Come:** contatori per valore $\rightarrow$ ricostruzione.
- **Costo:** $\Theta(n+b)$ tempo, $\Theta(b)$ spazio.

4.2 BucketSort

Dettaglio

Assunzione. Record con chiave intera in $[0, b-1]$. **Idea.** Metti ogni elemento nel bucket (coda) corrispondente alla chiave; poi concatena i bucket in ordine.

Complessità. $\Theta(n+b)$ tempo (con code efficienti), $\Theta(n+b)$ spazio. **Proprietà.** Stabile se ogni bucket è una coda FIFO.

Mappa mentale

- **Quando:** chiavi discrete piccole; serve stabilità.
- **Come:** distribuisce nei bucket $\rightarrow$ leggi bucket in ordine.
- **Costo:** $\Theta(n+b)$.

4.3 RadixSort (LSD, base B)

Dettaglio

Idea. Ordina ripetutamente per cifra (dalla meno significativa), usando un bucket sort stabile sulla cifra in posizione t .

Algorithm 10 RadixSort (LSD)

```
1: procedure RADIXSORT( $A, B$ )
2:    $t \leftarrow 0$ 
3:   while esiste una chiave  $k$  con  $\lfloor k/B^t \rfloor \neq 0$  do
4:     BUCKETSORTBYDIGIT( $A, B, t$ ) ▷ stabile
5:      $t \leftarrow t + 1$ 
```

Complessità. Se le chiavi hanno d cifre in base B , allora $\Theta(d(n+B))$. **Note.** La stabilità del sort per cifra è essenziale.

Mappa mentale

- **Quando:** chiavi intere lunghe ma con cifre “poche” (o d piccolo).
- **Come:** ripeti bucket sort stabile per cifra LSD $\rightarrow$ MSD.
- **Costo:** $\Theta(d(n + B))$.

5 Visite di grafi e alberi

Assumiamo un grafo $G = (V, E)$ rappresentato con liste di adiacenza.

5.1 BFS (visita in ampiezza)

Dettaglio

Obiettivo. Visitare i vertici a distanza crescente da una sorgente s ; costruisce un albero di visita. **Struttura dati.** Coda FIFO.

Algorithm 11 BFS

```
1: procedure BFS( $G = (V, E), s$ )
2:   Sia  $vis$  un vettore indicizzato da  $V$ 
3:   Sia  $pred$  un vettore indicizzato da  $V$  (opzionale)
4:   for all  $v \in V$  do
5:      $vis[v] \leftarrow \text{false}$ ;  $pred[v]$  non definito
6:    $Q \leftarrow$  coda vuota
7:    $vis[s] \leftarrow \text{true}$ ; ENQUEUE( $Q, s$ )
8:   while  $Q \neq \emptyset$  do
9:      $u \leftarrow$  DEQUEUE( $Q$ )
10:    for all  $v$  adiacente a  $u$  do
11:      if  $vis[v] = \text{false}$  then
12:         $vis[v] \leftarrow \text{true}$ ;  $pred[v] \leftarrow u$ 
13:        ENQUEUE( $Q, v$ )
14:   return  $pred$ 
```

Complessità. $\Theta(|V| + |E|)$. **Uso tipico.** Cammini minimi in grafi non pesati (distanza in numero di archi).

Mappa mentale

- **Coda FIFO** $\rightarrow$ livelli di distanza.
- **Risultato:** albero BFS + predecessori.
- **Costo:** $\Theta(n + m)$.

5.2 DFS (visita in profondità)

Dettaglio

Obiettivo. Esplorare andando “il più possibile in profondità” prima di tornare indietro. **Struttura dati.** Stack implicito (ricorsione) o esplicito.

Algorithm 12 DFS ricorsiva

```
1: procedure DFS( $G = (V, E), s$ )
2:   Sia  $vis$  un vettore indicizzato da  $V$ 
3:   Sia  $pred$  un vettore indicizzato da  $V$  (opzionale)
4:   for all  $v \in V$  do
5:      $vis[v] \leftarrow \mathbf{false}$ ;  $pred[v]$  non definito
6:   DFSVISIT( $G, s, vis, pred$ )
7:   return  $pred$ 
8: procedure DFSVISIT( $G, u, vis, pred$ )
9:    $vis[u] \leftarrow \mathbf{true}$ 
10:  for all  $v$  adiacente a  $u$  do
11:    if  $vis[v] = \mathbf{false}$  then
12:       $pred[v] \leftarrow u$ 
13:      DFSVISIT( $G, v, vis, pred$ )
```

Complessità. $\Theta(|V| + |E|)$. **Uso tipico.** Componenti connesse, rilevamento cicli (con opportune marcature), ordinamento topologico (su DAG), ecc.

Mappa mentale

- **Stack** (ricorsione) $\rightarrow$ esplora profondità.
- **Risultato:** albero DFS + predecessori.
- **Costo:** $\Theta(n + m)$.

5.3 Visite di alberi (pre/in/post-order, level-order)

Dettaglio

Per alberi radicati o alberi binari, le visite classiche sono:

- **Pre-order:** visita nodo, poi ricorsivamente i sottoalberi.
- **In-order** (binario): sinistro, nodo, destro.
- **Post-order:** sottoalberi, poi nodo.
- **Level-order:** BFS sui livelli (coda).

Sono tutte $\Theta(n)$ in tempo, con spazio $\mathcal{O}(h)$ per ricorsione (altezza h) o $\mathcal{O}(n)$ nel caso peggiore per lo stack esplicito.

Mappa mentale

- **Pre:** nodo $\rightarrow$ figli.
- **In** (binario): sx $\rightarrow$ nodo $\rightarrow$ dx.
- **Post:** figli $\rightarrow$ nodo.
- **Level:** coda (BFS).

6 Minimo albero ricoprente (MST)

Assumiamo grafo non orientato connesso e pesato $G = (V, E, \omega)$.

6.1 Kruskal

Dettaglio

Idea greedy. Ordina gli archi per peso crescente; aggiungi un arco se non crea ciclo (cioè se collega due componenti diverse). **Struttura dati.** Union-Find (Disjoint Set Union) per gestire le componenti.

Algorithm 13 Kruskal (con Union-Find)

```
1: procedure KRUSKAL( $G = (V, E, \omega)$ )
2:   ordina gli archi  $E$  per peso crescente
3:   for all  $v \in V$  do
4:     MAKESET( $v$ )
5:    $T \leftarrow (V, \emptyset)$ 
6:   for all  $(u, v) \in E$  in ordine do
7:      $T_u \leftarrow \text{FIND}(u)$ 
8:      $T_v \leftarrow \text{FIND}(v)$ 
9:     if  $T_u \neq T_v$  then
10:       $T \leftarrow T \cup \{(u, v)\}$ 
11:      UNION( $T_u, T_v$ )
12:   return  $T$ 
```

Complessità. Ordinamento $\mathcal{O}(m \log m)$. Union-Find quasi lineare: $\mathcal{O}(m \alpha(n))$. Totale tipico $\mathcal{O}(m \log m)$.

Mappa mentale

- **Greedy:** prendi archi leggeri.
- **Vincolo:** no cicli $\rightarrow$ DSU.
- **Costo:** $\mathcal{O}(m \log m)$.

6.2 Prim

Dettaglio

Idea greedy. Parti da un vertice qualsiasi s e fai crescere l'albero aggiungendo ogni volta l'arco più leggero che collega l'albero costruito a un vertice esterno.

Implementazione (versione della dispensa). Mantieni per ogni vertice esterno un valore $d[v]$ (peso minimo di un arco che lo connette all'albero) e un *vicino* $[v]$ che realizza tale minimo; ripeti scegliendo il vertice esterno con d minimo.

Algorithm 14 Prim (grafo connesso non orientato $G = (V, E, \omega)$)

```
1: procedure PRIM( $G = (V, E, \omega)$ )
2:   Siano vicino e d due array con insieme di indici  $V$ 
3:   for all  $v \in V$  do
4:      $d[v] \leftarrow \infty$ 
5:    $T_V \leftarrow \emptyset$ ;  $E_T \leftarrow \emptyset$ 
6:   repeat
7:      $y \leftarrow$  elemento di  $V \setminus T_V$  con valore di  $d[y]$  minimo
8:      $T_V \leftarrow T_V \cup \{y\}$ 
9:     if  $d[y] \neq \infty$  then ▷ falsa solo nella prima iterazione
10:       $x \leftarrow \text{vicino}[y]$ 
11:       $E_T \leftarrow E_T \cup \{(x, y)\}$ 
12:      for all  $(y, z) \in E$  do
13:        if  $z \notin T_V$  and  $\omega(y, z) < d[z]$  then
14:           $d[z] \leftarrow \omega(y, z)$ 
15:           $\text{vicino}[z] \leftarrow y$ 
16:    until  $V \setminus T_V = \emptyset$ 
17:  return  $T \leftarrow (T_V, E_T)$ 
```

Complessità. Con scansione lineare del minimo in C : $\Theta(n^2 + m) = \Theta(n^2)$ nel caso denso. Con una PQ (min-heap) per selezionare il minimo: $\mathcal{O}(m \log n)$ (liste di adiacenza).

Mappa mentale

- **Greedy:** espandi un solo albero.
- **Chiavi:** $d[v]$ = miglior collegamento all'albero.
- **Selezione min:** scansione in C oppure PQ (min-heap).
- **Costo:** $\Theta(n^2)$ (scansione) o $\mathcal{O}(m \log n)$ (PQ).
- **Note:** diverso dall'albero dei cammini minimi.

7 Cammini minimi nei grafi

7.1 Dijkstra (da una sorgente)

Dettaglio

Assunzione. Pesi non negativi. **Idea.** Mantieni stime $D[v]$ della distanza da s ; ripeti: scegli il vertice candidato con D minimo e rilassa i suoi archi in uscita.

Algorithm 15 Dijkstra (schema classico)

```
1: procedure DIJKSTRA( $G = (V, E, \omega), s$ )
2:   Sia  $D$  un vettore indicizzato da  $V$ 
3:    $D[s] \leftarrow 0$ 
4:   for all  $v \in V \setminus \{s\}$  do
5:      $D[v] \leftarrow \infty$ 
6:    $C \leftarrow V$ 
7:   while  $C \neq \emptyset$  do
8:      $u \leftarrow$  elemento di  $C$  con  $D[u]$  minima
9:      $C \leftarrow C \setminus \{u\}$ 
10:    for all  $(u, v) \in E$  do
11:      if  $D[u] + \omega(u, v) < D[v]$  then
12:         $D[v] \leftarrow D[u] + \omega(u, v)$ 
13:         $pred[v] \leftarrow u$  ▷ opzionale: per ricostruire i cammini
14:  return  $D$  ▷ e  $pred$  se mantenuto
```

Complessità. Se il minimo in C si trova con scansione: $\Theta(n^2 + m) = \Theta(n^2)$ nel caso denso. Con una PQ (min-heap) per selezionare il minimo: $\mathcal{O}(m \log n)$ (liste di adiacenza). **Output.** D (distanze) e, se mantenuto, $pred$ per ricostruire i cammini (a ritroso).

Mappa mentale

- **Richiede:** pesi ≥ 0 .
- **Meccanismo:** estrai min distanza $\rightarrow$ rilassa archi.
- **Implementazione:** min in C con scansione **oppure** PQ (min-heap).
- **Costo:** $\Theta(n^2)$ (scansione) o $\mathcal{O}(m \log n)$ (PQ).

7.2 Bellman–Ford (da una sorgente)

Dettaglio

Assunzione. Nessun ciclo di peso negativo raggiungibile (in caso contrario le distanze minime non sono ben definite). **Idea.** Rilassa tutti gli archi ripetutamente per $n - 1$ iterazioni.

Algorithm 16 Bellman–Ford

```
1: procedure BELLMANFORD( $G, s$ )
2:   for all  $v \in V$  do
3:      $D[v] \leftarrow \infty$ 
4:    $D[s] \leftarrow 0$ 
5:   for  $k \leftarrow 1$  to  $n - 1$  do
6:     for all  $(u, v) \in E$  do
7:       if  $D[u] + \omega(u, v) < D[v]$  then
8:          $D[v] \leftarrow D[u] + \omega(u, v)$ 
9:  return  $D$ 
```

Complessità. $\Theta(nm)$. **Nota pratica.** Con un'ulteriore iterazione si può rilevare l'esistenza di cicli negativi (se una distanza diminuisce ancora).

Mappa mentale

- **Pro:** gestisce pesi negativi.
- **Contro:** lento $\Theta(nm)$.
- **Come:** ripeti $n - 1$ volte: rilassa tutti gli archi.

7.3 Floyd–Warshall (tutte le coppie)

Dettaglio

Assunzione. Nessun ciclo di peso negativo. **Idea (DP).** $D[i, j]$ = miglior distanza da v_i a v_j usando solo intermedi tra $\{v_1, \dots, v_k\}$; aumenta gradualmente l'insieme di intermedi.

Algorithm 17 Floyd–Warshall (distanze)

```

1: procedure FLOYDWARSHALL( $G$ )
2:   inizializza  $D$  con 0 in diagonale,  $\omega(i, j)$  se arco, altrimenti  $\infty$ 
3:   for  $k \leftarrow 1$  to  $n$  do
4:     for  $i \leftarrow 1$  to  $n$  do
5:       for  $j \leftarrow 1$  to  $n$  do
6:         if  $D[i, k] + D[k, j] < D[i, j]$  then
7:            $D[i, j] \leftarrow D[i, k] + D[k, j]$ 
8:   return  $D$ 

```

Complessità. $\Theta(n^3)$ tempo, $\Theta(n^2)$ spazio. **Ricostruzione cammino.** Si può mantenere una matrice di “predecessori/penultimi” per ricostruire i cammini minimi.

Mappa mentale

- **Obiettivo:** tutte le coppie.
- **DP:** prova intermedi uno a uno.
- **Costo:** $\Theta(n^3)$.

8 Algoritmi non deterministici per problemi NP

8.1 Schema generale “indovina e verifica”

Dettaglio

Molti problemi NP hanno il seguente schema:

1. **Indovina** (nondeterministicamente) un certificato c di dimensione polinomiale.
2. **Verifica** in tempo polinomiale che c sia una soluzione valida.

Su una macchina non deterministica questo porta a tempo polinomiale; nella pratica, lo schema corrisponde spesso a *backtracking* (deterministico) con potenziale esplosione esponenziale nel caso peggiore.

8.2 Partizione (Partition)

Dettaglio

Problema. Dato un multinsieme di interi S , decidere se esiste una scelta di elementi con somma uguale alla somma dei non scelti. **Nondeterminismo.** Indovina un bit $b[i] \in \{0, 1\}$ per ogni elemento, poi calcola le due somme e confronta.

Algorithm 18 Partizione (nondeterministico: indovina + verifica)

```
1: procedure PARTIZIONE( $S[1 \dots n]$ )
2:   for  $i \leftarrow 1$  to  $n$  do
3:      $b[i] \leftarrow$  indovina un valore in  $\{0, 1\}$ 
4:    $somma1 \leftarrow 0$ ;  $somma0 \leftarrow 0$ 
5:   for  $i \leftarrow 1$  to  $n$  do
6:     if  $b[i] = 1$  then
7:        $somma1 \leftarrow somma1 + S[i]$ 
8:     else
9:        $somma0 \leftarrow somma0 + S[i]$ 
10:  return ( $somma1 = somma0$ )
```

Mappa mentale

- **Certificato:** bitmask $b[1..n]$.
- **Verifica:** calcola due somme e confronta.
- **Tempo verifica:** $\Theta(n)$.

8.3 Clique di dimensione k **Dettaglio**

Problema. Dato $G = (V, E)$ e k , decidere se esiste un insieme $V' \subseteq V$ con $|V'| = k$ tale che ogni coppia di vertici in V' sia collegata (clique). **Nondeterminismo.** Indovina V' (ad esempio scegliendo un bit per ogni vertice), controlla $|V'| = k$ e poi verifica tutti i lati tra coppie.

Algorithm 19 Clique(G, k) (nondeterministico: indovina + verifica)

```
1: procedure CLIQUE( $G, k$ )
2:    $V' \leftarrow \emptyset$ 
3:   for all  $v \in V$  do
4:      $b \leftarrow$  indovina un valore in  $\{0, 1\}$ 
5:     if  $b = 1$  then
6:        $V' \leftarrow V' \cup \{v\}$ 
7:   if  $|V'| \neq k$  then
8:     return false
9:   for all coppie distinte  $u, v \in V'$  do
10:    if  $(u, v) \notin E$  then
11:      return false
12:  return true
```

Tempo verifica. Se $|V'| = k$, controllo coppie $\Theta(k^2)$ (con test di adiacenza $\mathcal{O}(1)$ usando matrice di adiacenza).

Mappa mentale

- **Certificato:** sottoinsieme V' di k vertici.
- **Verifica:** per ogni coppia (u, v) in V' controlla che $(u, v) \in E$.
- **Tempo verifica:** $\Theta(k^2)$.